

Ex.No.3 – Wireshark

Date:

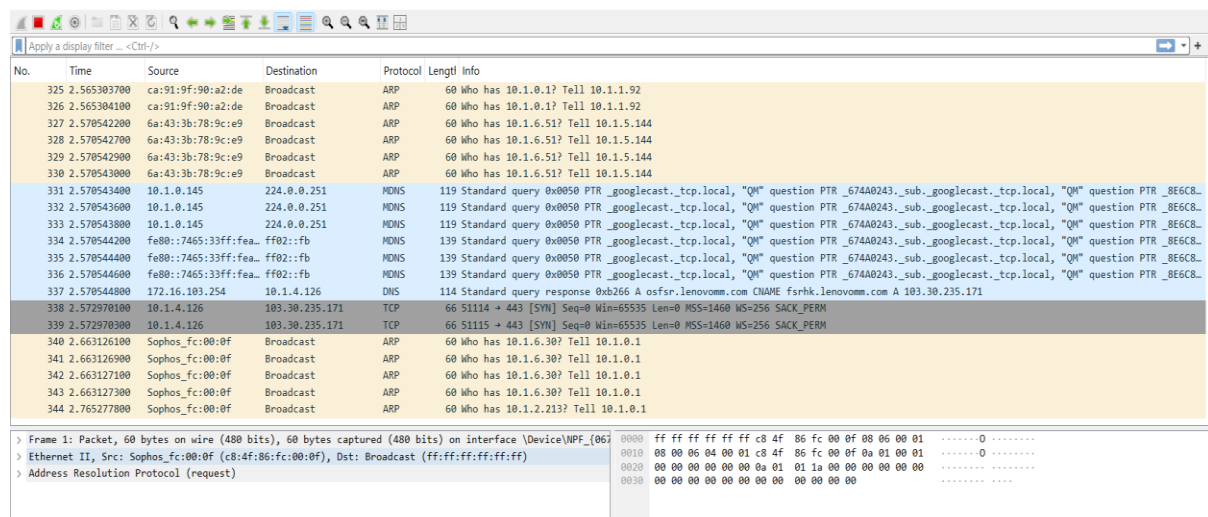
Password Capturing

Wireshark can capture not only passwords but any type of information transmitted over the network: usernames, email addresses, personal information, etc. As long as we can capture network traffic, Wireshark can sniff passing passwords.

In sniffing can include passwords for various protocols such as HTTP, FTP, Telnet, etc. the captured data can be used to troubleshoot network problems, but can also be used maliciously to gain unauthorized access to sensitive information.

So, here we will see how we can capture the password using the Wireshark network capture analyzer. and see the outputs of the following steps.

Step 1: First of all, open your Wireshark tool in your window or in Linux virtual machine. and start capturing the network. suppose I am capturing my wireless fidelity.



The image shows a Wireshark network traffic capture. The top pane displays a list of captured packets with columns for No., Time, Source, Destination, Protocol, Length, and Info. The bottom pane shows the details of the selected packet (No. 338), including Ethernet II, Internet Protocol Version 4, and Hypertext Transfer Protocol.

No.	Time	Source	Destination	Protocol	Length	Info
325	2.565303700	ca:91:9f:90:a2:de	Broadcast	ARP	60	Who has 10.1.0.1? Tell 10.1.1.92
326	2.565304100	ca:91:9f:90:a2:de	Broadcast	ARP	60	Who has 10.1.0.1? Tell 10.1.1.92
327	2.570542200	6a:43:3b:78:9c:e9	Broadcast	ARP	60	Who has 10.1.6.51? Tell 10.1.5.144
328	2.570542700	6a:43:3b:78:9c:e9	Broadcast	ARP	60	Who has 10.1.6.51? Tell 10.1.5.144
329	2.570542900	6a:43:3b:78:9c:e9	Broadcast	ARP	60	Who has 10.1.6.51? Tell 10.1.5.144
330	2.570543000	6a:43:3b:78:9c:e9	Broadcast	ARP	60	Who has 10.1.6.51? Tell 10.1.5.144
331	2.570543400	10.1.0.145	224.0.0.251	MDNS	119	Standard query 0x0050 PTR _googlecast._tcp.local, "QM" question PTR _674A0243._sub._googlecast._tcp.local, "QM" question PTR _8E6C8._sub._googlecast._tcp.local
332	2.570543600	10.1.0.145	224.0.0.251	MDNS	119	Standard query 0x0050 PTR _googlecast._tcp.local, "QM" question PTR _674A0243._sub._googlecast._tcp.local, "QM" question PTR _8E6C8._sub._googlecast._tcp.local
333	2.570543800	10.1.0.145	224.0.0.251	MDNS	119	Standard query 0x0050 PTR _googlecast._tcp.local, "QM" question PTR _674A0243._sub._googlecast._tcp.local, "QM" question PTR _8E6C8._sub._googlecast._tcp.local
334	2.570544200	fe80::7465:33ff:fea::ff02::fb	ff02::fb	MDNS	139	Standard query 0x0050 PTR _googlecast._tcp.local, "QM" question PTR _674A0243._sub._googlecast._tcp.local, "QM" question PTR _8E6C8._sub._googlecast._tcp.local
335	2.570544400	fe80::7465:33ff:fea::ff02::fb	ff02::fb	MDNS	139	Standard query 0x0050 PTR _googlecast._tcp.local, "QM" question PTR _674A0243._sub._googlecast._tcp.local, "QM" question PTR _8E6C8._sub._googlecast._tcp.local
336	2.570544600	fe80::7465:33ff:fea::ff02::fb	ff02::fb	MDNS	139	Standard query 0x0050 PTR _googlecast._tcp.local, "QM" question PTR _674A0243._sub._googlecast._tcp.local, "QM" question PTR _8E6C8._sub._googlecast._tcp.local
337	2.570544800	172.16.103.254	10.1.4.126	DNS	114	Standard query response 0xb266 A osfsr.lenovomm.com CNAME fsrhhk.lenovomm.com A 103.30.235.171
338	2.572970100	10.1.4.126	103.30.235.171	TCP	66	51114 → 443 [SYN] Seq=0 Win=65535 Len=0 MSS=1460 WS=256 SACK_PERM
339	2.572970300	10.1.4.126	103.30.235.171	TCP	66	51115 → 443 [SYN] Seq=0 Win=65535 Len=0 MSS=1460 WS=256 SACK_PERM
340	2.663126100	Sophos_fc:00:0f	Broadcast	ARP	60	Who has 10.1.6.30? Tell 10.1.0.1
341	2.663126900	Sophos_fc:00:0f	Broadcast	ARP	60	Who has 10.1.6.30? Tell 10.1.0.1
342	2.663127100	Sophos_fc:00:0f	Broadcast	ARP	60	Who has 10.1.6.30? Tell 10.1.0.1
343	2.663127300	Sophos_fc:00:0f	Broadcast	ARP	60	Who has 10.1.6.30? Tell 10.1.0.1
344	2.765277800	Sophos_fc:00:0f	Broadcast	ARP	60	Who has 10.1.2.213? Tell 10.1.0.1

Frame 1: Packet, 60 bytes on wire (480 bits), 60 bytes captured (480 bits) on interface \Device\NPF...
> Ethernet II, Src: Sophos_fc:00:0f (c8:4f:86:fc:00:0f), Dst: Broadcast (ff:ff:ff:ff:ff:ff)
> Address Resolution Protocol (request)

Step 2: After starting the packet capturing we will go to the website and login the credential on that website as you can see in the image.

login page

Not secure | testphp.vulnweb.com/login.php

acunetix acuart

TEST and Demonstration site for Acunetix Web Vulnerability Scanner

home | categories | artists | disclaimer | your cart | guestbook | AJAX Demo

search art

go

Browse categories

Browse artists

Your cart

Signup

Your profile

Our guestbook

AJAX Demo

If you are already registered please enter your login information below:

Username : Tonystark_44

Password :

login

You can also [signup here](#).

Signup disabled. Please use the username **test** and the password **test**.

Step 3: Now after completing the login credential we will go and capture the password in Wireshark. for that we have to use some filter that helps to find the login credential through the packet capturing.

Step 4: Wireshark has captured some packets but we specifically looking for HTTP packets. so in the display filter bar we use some command to find all the captured HTTP packets. as you can see in the below image the green bar where we apply the filter.

http

No.	Destination	Protocol	Length	Info
1	127.0.0.1	HTTP	756	GET / HTTP/1.1
2	127.0.0.1	HTTP	44	HTTP/1.0 200 OK (text/html)
3	127.0.0.1	HTTP	723	GET / HTTP/1.1
4	127.0.0.1	HTTP	44	HTTP/1.0 200 OK (text/html)
5	127.0.0.1	HTTP	649	GET /favicon.ico HTTP/1.1
6	127.0.0.1	HTTP	649	GET /favicon.ico HTTP/1.1
7	127.0.0.1	HTTP	936	POST /login HTTP/1.1 (application/x-www-form-urlencoded)
8	127.0.0.1	HTTP	44	HTTP/1.0 200 OK (text/html)
9	127.0.0.1	HTTP	654	GET /favicon.ico HTTP/1.1
10	127.0.0.1	HTTP	654	GET /favicon.ico HTTP/1.1

```

> Frame 707: Packet, 756 bytes on wire (6048 bits), 756 bytes captured (6048 bits) on interface \Device\NPF...
> Null/Loopback
> Internet Protocol Version 4, Src: 127.0.0.1, Dst: 127.0.0.1
> Transmission Control Protocol, Src Port: 61231, Dst Port: 8080, Seq: 1, Ack: 1, Len: 712
> Hypertext Transfer Protocol
0000  02 00 00 00 45 00 02 f0 27 e4 40 00 80 06 d2 21  ....E:
0010  7f 00 00 01 7f 00 00 01 ef 2f 1f 40 3f 68 4d bd  ....
0020  b2 b7 4d ad 50 18 00 ff 4d 9a 00 00 47 45 54 20  ..M-P-
0030  2f 20 48 54 54 50 2f 31 2e 31 0d 0a 48 6f 73 74  / HTTP,
0040  3a 20 31 32 37 2e 30 2e 30 2e 31 3a 38 30 30 30  : 127.0.
0050  0d 0a 43 6f 6e 6e 65 63 74 69 6f 6e 3a 20 6b 65  ..Conn
0060  65 70 2d 61 6c 69 76 65 0d 0a 73 65 63 2d 63 68  ep=ali
0070  2d 75 61 3a 20 22 43 68 72 6f 6d 69 75 6d 22 3b  ~ua: "I
0080  76 3d 22 31 35 32 22 2c 20 22 4e 6f 74 3f 41 5f  v="152"
0090  42 72 61 6e 64 22 3b 76 3d 22 32 34 22 2c 20 22  Brand".
00a0  4d 69 63 72 6f 73 6f 66 74 20 45 64 67 65 22 3b  Microso
00b0  76 3d 22 31 35 32 22 0d 0a 73 65 63 2d 63 68 2d  v="152"
00c0  75 61 2d 6d 6f 62 69 6c 65 3a 20 3f 30 0d 0a 73  ua-mob:
00d0  65 63 2d 63 68 2d 75 61 2d 70 6c 61 74 66 6f 72  ec-ch-i

```

Step 5: So there are some HTTP packets are captured but we specifically looking for form data that the user submitted to the website. for that, we have a separate filter

As we know that there are main two methods used for submitting form data from web pages like login forms to the server. the methods are-

- GET
- POST

Step 6: So firstly for knowing the credential we use the first method and apply the filter for the GET methods as you can see below.
http.request.method == "GET"

```
Wireshark - Packet 1656 - Adapter for loopback traffic capture

> Frame 1656: Packet, 936 bytes on wire (7488 bits), 936 bytes captured (7488 bits) on interface \Device\NPF_{...}_id 0
> Null/Loopback
> Internet Protocol Version 4, Src: 127.0.0.1, Dst: 127.0.0.1
> Transmission Control Protocol, Src Port: 54941, Dst Port: 8000, Seq: 1, Ack: 1, Len: 892
▼ Hypertext Transfer Protocol
  > POST /login HTTP/1.1\r\n
    Host: localhost:8000\r\n
    Connection: keep-alive\r\n
  > Content-Length: 42\r\n
    Cache-Control: max-age=0\r\n
    sec-ch-ua: "Chromium";v="152", "Not?A_Brand";v="24", "Microsoft Edge";v="152"\r\n
    sec-ch-ua-mobile: ?0\r\n
    sec-ch-ua-platform: "Windows"\r\n
    Upgrade-Insecure-Requests: 1\r\n
    Content-Type: application/x-www-form-urlencoded\r\n
    User-Agent: Mozilla/5.0 (Windows NT 10.0; Win64; x64) AppleWebKit/537.36 (KHTML, like Gecko) Chrome/152.0.0.0 Safari/537.36 Edg/152.0.0.0\r\n

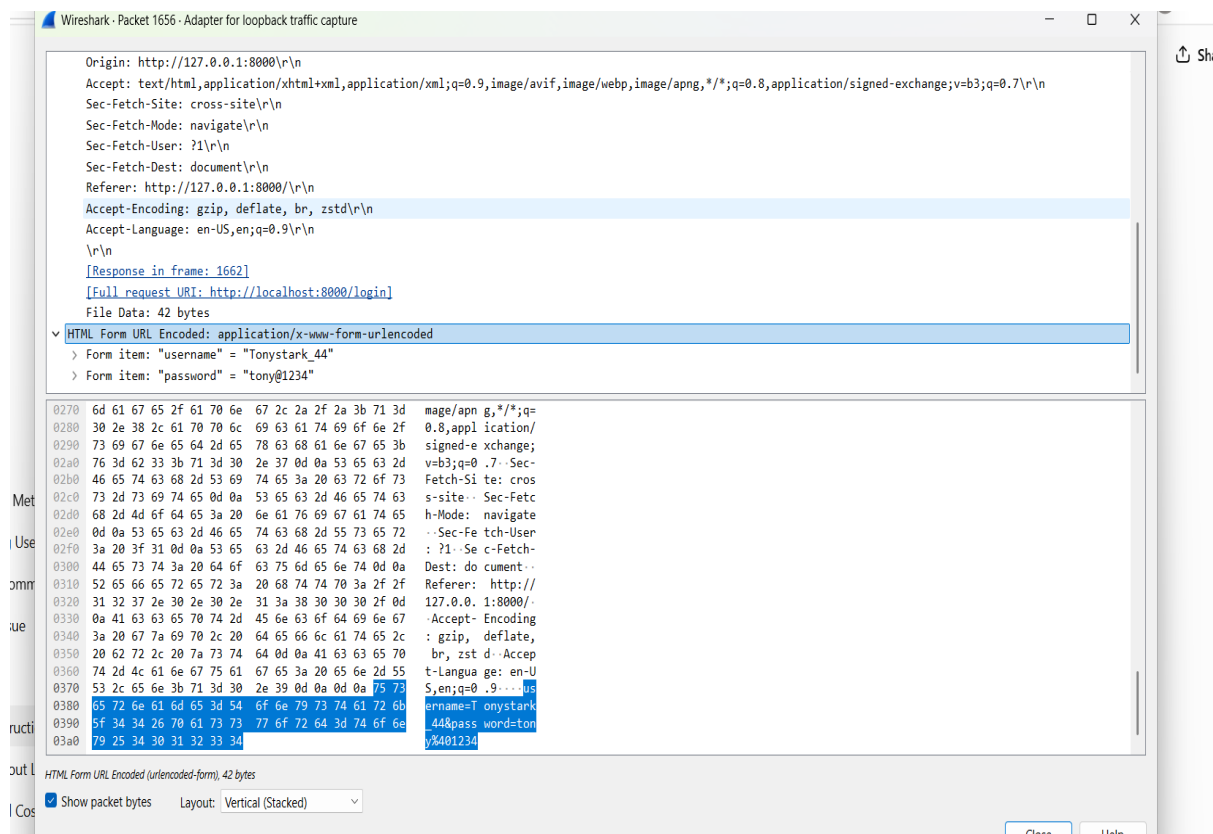
0270  6d 61 67 65 2f 61 70 6e 67 2c 2a 2f 2a 3b 71 3d  maged/apn g,/*;q=
0280  30 2e 38 2c 61 70 70 6c 69 63 61 74 69 6f 6e 2f  0.8,application/
0290  73 69 67 6e 65 64 2d 65 78 63 68 61 6e 67 65 3b  signed-exchange;
02a0  76 3d 62 33 3b 71 3d 30 2e 37 0d 0a 53 65 63 2d  v=b3;q=0.7;Sec-
02b0  46 65 74 63 68 2d 53 69 74 65 3a 20 63 72 6f 73  Fetch-Site: cros
02c0  73 2d 73 69 74 65 0d 0a 53 65 63 2d 46 65 74 63  s-site: Sec-Fetc
02d0  68 2d 4d 6f 64 65 3a 20 6e 61 76 69 67 61 74 65  h-Mode: navigate
02e0  0d 0a 53 65 63 2d 46 65 74 63 68 2d 55 73 65 72  ..Sec-Fetch-User
02f0  3a 20 3f 31 0d 0a 53 65 63 2d 46 65 74 63 68 2d  : ?1..Sec-Fetch-
0300  44 65 73 74 3a 20 64 6f 63 75 6d 65 6e 74 0d 0a  Dest: document..
0310  52 65 66 65 72 65 72 3a 20 68 74 74 70 3a 2f 2f  Referer: http://
0320  31 32 37 2e 30 2e 30 2e 31 3a 38 30 30 30 2f 0d  127.0.0.1:8000/.
0330  0a 41 63 63 65 70 74 2d 45 6e 63 6f 64 69 6e 67  .Accept-Encoding
0340  3a 20 67 7a 69 70 2c 20 64 65 66 6c 61 74 65 2c  : gzip, deflate,
0350  20 62 72 2c 20 7a 73 74 64 0d 0a 41 63 63 65 70  br, zstd..Accept
0360  74 2d 4c 61 6e 67 75 61 67 65 3a 20 65 6e 2d 55  t-Language: en-U
0370  53 2c 65 6e 3b 71 3d 30 2e 39 0d 0a 0d 0a 75 73  S,en;q=0.9...us
0380  65 72 6e 61 6d 65 3d 54 6f 6e 79 73 74 61 72 6b  ername=Tony Stark
0390  5f 34 34 26 70 61 73 73 77 6f 72 64 3d 74 6f 6e  _44&password=ton
03a0  79 25 34 30 31 32 33 34  y%401234

Key (urlencoded-form.key), 8 bytes
```

GET method

As you can see in the image there are two packets where the login page was requested with a GET request as well, but there is no form data submitted with a GET request.

Step 7: Now after checking the GET method if we didn't find the form data, then we will try the POST method for that we will apply the filter on Wireshark as you can see.
http.request.method == "POST"



As you can see we have a packet with form data click on the packet with user info and the application URL encoded. and click on the down-HTML form URL Encoded where the login credential is found. login credential as it is the same that we filed on the website in step 2.

Form item: "uname" = "Tonystark_44"

Form item: "pass" = "tony@1234"